EEE3095S Practical 1A

STM32 GPIO, Timers, and Interrupts

Sibahle Mndebele, MNDSIB016
Adam Foflonker, FFLADA001

August 14, 2026

1 Hardware Orientation

1. GPIO pins connected to the 8 user LEDs: `PB0, PB1, PB2, PB3, PB4, PB5, PB6, PB7`
2. GPIO pins connected to the 4 push buttons: `PA0, PA1, PA2, PA3`
3. GPIO pins connected to the 8 MHz crystal: `Y1 and XTAL`
4. Default system clock frequency using the internal HSI oscillator: `8 MHz`
5. Purpose of the loopback header P2: `The loopback header P2 provides connections between selected GPIO/signal pins, allowing signals generated or measured on one part of the board to be routed back to another pin for testing and verification.`

2 Task 1: Timer Configuration and Verification

2.1 Predictions

The system uses the internal HSI oscillator, giving a system clock frequency of 8 MHz. A prescaler of 7999 was selected to divide the clock to a convenient timer frequency of 1 kHz. An ARR value of 999 was then expected to produce a 1 Hz timer update.

- System clock frequency: 8 MHz
- Expected prescaler value: 7999
- Expected ARR value: 999
- Expected interrupt period: 1 s

2.2 Calculations

The timer clock frequency is calculated using:

$$\begin{aligned} f_{\text{timer}} &= \frac{f_{\text{sys}}}{PSC + 1} \\ &= \frac{8\,000\,000}{7999 + 1} \\ &= \boxed{1000 \text{ Hz}} \end{aligned}$$

The ARR value is calculated using:

$$
\begin{aligned}
f_{\text{update}} &= \frac{f_{\text{timer}}}{ARR+1} \\
1 &= \frac{1000}{ARR+1} \\
ARR+1 &= 1000 \\
ARR &= \boxed{999}.
\end{aligned}
$$

Therefore, the expected timer interrupt period is:

$$
\begin{aligned}
T &= \frac{1}{f_{\text{update}}} \\
&= \frac{1}{1} \\
&= \boxed{1\ \text{s}}.
\end{aligned}
$$

2.3 Measurement and Verification

- Measured period from the oscilloscope: 1 second
- Comparison with your expected period: No difference at all.

3 Task 2: LED Control with Timer Interrupt

Confirm your running light video is in your submission folder, or link it here: A demonstration video of the running-light sequence is available in the Google Drive submission folder.

4 Task 3: Button Input and Software Debounce

4.1 Records

- Expected switch bounce duration: Around 50ms
- Maximum bounce duration measured on the oscilloscope: Around 10ms
- Chosen software debounce delay time: 50ms
- Formula to calculate the ARR value from the desired period in milliseconds:
- The timer prescaler is kept fixed at 7999. The ARR value for a desired timer period can be calculated from

$$
T = \frac{(PSC+1)(ARR+1)}{f_{\text{sys}}}.
$$

Rearranging for ARR gives

$$
ARR = \frac{T f_{\text{sys}}}{PSC+1} - 1.
$$

When the desired period is given in milliseconds, the period must first be converted to seconds:

$$T = \frac{T_{\text{ms}}}{1000}.$$

Therefore,

$$ARR = \frac{(T_{\text{ms}}/1000) f_{\text{sys}}}{PSC + 1} - 1.$$

For this practical, $f_{\text{sys}} = 8$ MHz and $PSC = 7999$, giving

$$\begin{aligned} ARR &= \frac{(T_{\text{ms}}/1000)(8\,000\,000)}{8000} - 1 \\ &= T_{\text{ms}} - 1. \end{aligned}$$

For a 1 s period,

$$\begin{aligned} T_{\text{ms}} &= 1000 \text{ ms} \\ ARR &= 1000 - 1 \\ &= \boxed{999}. \end{aligned}$$

For a 0.5 s period,

$$\begin{aligned} T_{\text{ms}} &= 500 \text{ ms} \\ ARR &= 500 - 1 \\ &= \boxed{499}. \end{aligned}$$

Therefore, PA0 changes the ARR register between 999 and 499 while keeping the prescaler fixed at 7999.

4.2 Question 4

When you update the ARR register dynamically, does the timer frequency change immediately mid-cycle, or does the timer wait for the next update event? Base your answer on the STM32 timer preload register behaviour.

Answer: The new ARR value is transferred to the active auto-reload register at the next update event because ARR preload is enabled. Therefore, the currently active timer period is not changed immediately in the middle of a counting cycle.

5 Task 4: Multi-Mode LED Control

5.1 State Machine Explanation

Mode 3 is implemented using three states: **SPARKLE_IDLE**, **SPARKLE_DISPLAY**, and **SPARKLE_TURN_OFF**.

SPARKLE_IDLE: The system waits in this state until the sparkle sequence should begin. The LEDs remain in their normal/off state. Once the required trigger or timing condition occurs, the state changes to **SPARKLE_DISPLAY**.

SPARKLE_DISPLAY: In this state, the sparkle pattern is displayed on the LEDs. The timer/counter controls how long the sparkle pattern is shown. Once the display period has elapsed, the state transitions to **SPARKLE_TURN_OFF**.

SPARKLE_TURN_OFF: This state turns the LEDs off and performs any required reset or cleanup for the sparkle sequence. After this, the state returns to **SPARKLE_IDLE**, where the system waits for the next sparkle event.

The overall state transition is:

SPARKLE_IDLE → SPARKLE_DISPLAY → SPARKLE_TURN_OFF → SPARKLE_IDLE

Each state is responsible for one specific part of the sparkle sequence, and transitions occur when the corresponding condition or timer event is reached.

5.2 Measurements

- Measured minimum LED-on time for Mode 3: 0.1 seconds
- Measured maximum LED-on time for Mode 3: 1.5 seconds

6 Task 5: ADC Interrupts and LED Level Meter

6.1 Records

- Predicted 12-bit ADC register value, in decimal, for a 2.0 V input: 2482
- Measured multimeter voltage at the moment the 5th LED turns on: 2.06V
- Calculation comparing your measured voltage against the theoretical threshold: 2.06 - 2.06 = 0.00, But if we're being more accurate then; 2.0625 - 2.06(reading from the voltmeter) = 0.0025V

6.2 Question

If a student accidentally sets the signal generator to output a 5.0 V DC signal into PA5, what physical damage might occur? Consult the I/O structure column for PA5 in the STM32F051x datasheet.

Applying 5.0 V to PA5 would exceed the permitted input voltage for the ADC input. This could cause excessive injection/protection-diode current and may result in malfunction or permanent damage to the PA5 input circuitry or the microcontroller.

7 Task 6: Software PWM via Timer Interrupts

7.1 Records

- Calculated Timer Prescaler and ARR values for exactly 10 kHz: $PSC = 7$, $ARR = 99$
- Measured total period of the signal: 60ms
- Measured positive pulse width: 18ms

7.2 Question

A project requires a 50 Hz signal with a duty cycle resolution of 1 percent. What is the exact timer interrupt frequency required? Show your calculation.

A 50 Hz signal has a period of:

$$\begin{aligned} T_{\text{PWM}} &= \frac{1}{f_{\text{PWM}}} \\ &= \frac{1}{50} \\ &= 0.02 \text{ s} \\ &= 20 \text{ ms}. \end{aligned}$$

A duty cycle resolution of 1% requires the PWM period to be divided into 100 equal intervals. Therefore, the timer must generate 100 interrupts during each PWM period.

The required timer interrupt frequency is therefore:

$$\begin{aligned} f_{\text{interrupt}} &= 50 \times 100 \\ &= \boxed{5000 \text{ Hz}}. \end{aligned}$$

Therefore, the exact timer interrupt frequency required is $\boxed{5 \text{ kHz}}$.